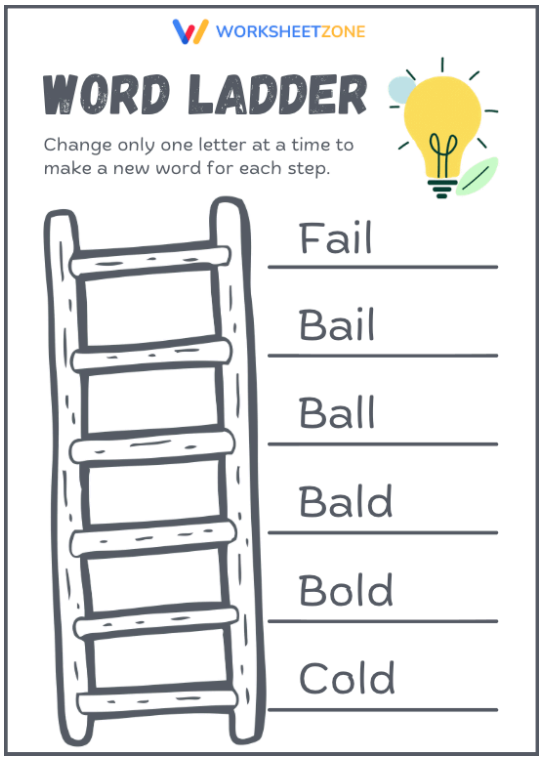
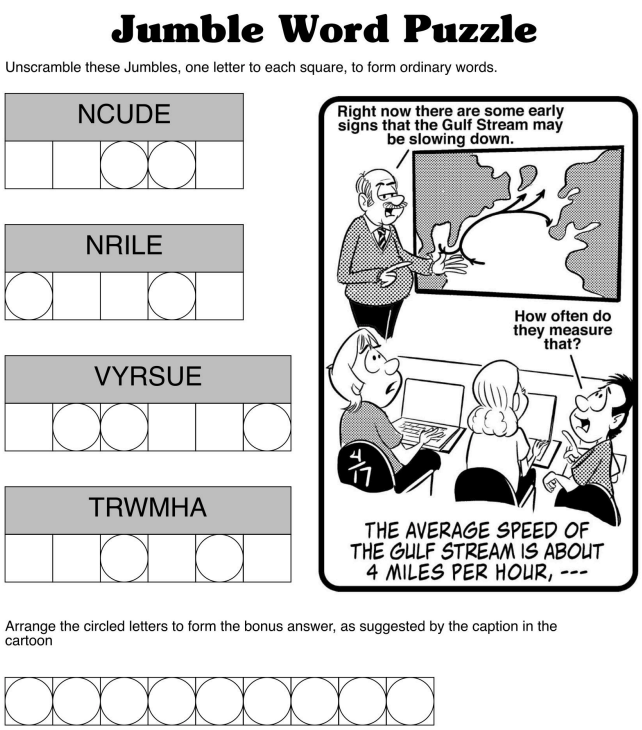
Object Design Document





Word Games
FLEX

Project Manager: Daylin Jackson
Team: Luke Hayden
Zamiyah Hill
Dillon McCants

1 Introduction **2**
1.1 Object Design Trade-offs 2
1.2 Interface Conventions 2
2 Design Specifications **2**
2.1 Use Case Realizations 2
2.2 Design Class Diagram 3
2.3 Major Patterns 3

1 Introduction

1.1 Object Design Trade-offs

In programming, developers must always make trade-offs. For example, a developer may have to choose between complexity and performance. A developer could choose to add more features to their project, but that may decrease performance so a choice must be made. Developers also have to choose whether to build systems themselves or use 3rd party ones. In our case for Word Games, we will use JavaFX to create the user interface (UI). It would take a lot longer to write a UI system from scratch compared to using JavaFX. It would also be very difficult to make it look as good as JavaFX can look.

1.2 Interface Conventions

Interface documentation guidelines and coding conventions. These include a list of rules that developers should use when designing and naming interfaces.

Remember that interfaces make your code more extensible and resilient to change. At a minimum, you should mention naming conventions for interfaces or what is implementing the interface.

More information found in the book that you can use to help with this section:

- Identifying Missing Attributes and Operations (Section 9.4.1)
- Specifying Type Signatures and Visibility (Section 9.4.2)
- Specifying Preconditions and Postconditions (Section 9.4.3)
- Specifying Invariants (Section 9.4.4)
- Inheriting Contracts (Section 9.4.5)

These conventions apply to every class and interface in Word Games. We keep the code readable for all four of us and make it possible to add a new game later without rewriting existing code. Wherever one object depends on another part of the system, it depends on an interface or the abstract *Game* class rather than a concrete class (for example, Game game = gameSystem.startGame(type, difficulty);), so implementations can change without breaking the code that calls them.

Naming Conventions

Element	Convention	Example
Packages	All lowercase, grouped by MVC layer	wordgames.model, wordgames.view, wordgames.controller
Classes	UpperCamelCase nouns, matching the names in the RAD object model	GameSystem, WordLadder, Puzzle, Score
Interfaces	UpperCamelCase adjective or capability name, no "I" prefix	Playable, GameObserver, PuzzleSource
Implementing classes	Named for what they are; the interface name describes what they can do	PuzzleBank implements PuzzleSource; JumbleController implements GameObserver
Abstract classes	Named after the general concept and declared abstract	Game (extended by Jumble and WordLadder)
Methods	lowerCamelCase verbs	submitAnswer(), scrambleWord()
Boolean methods	Start with is / has / can	isActive(), hasAttemptsLeft()
Controller event handlers	"on" + the event	onSubmit(), onViewScore(), onEndGame()
Fields and variables	lowerCamelCase, descriptive (no single letters except loop counters)	targetWord, attemptsLeft
Constants	UPPER_SNAKE_CASE, static final	MAX_ATTEMPTS, POINTS_PER_CORRECT
Enums	UpperCamelCase type, UPPER_SNAKE_CASE values	GameStatus.ACTIVE, Difficulty.HARD
FXML files and controls	File named after its view; fx:id = purpose + control type	JumbleView.fxml; submitButton, answerField

Core Interfaces

- **Playable**: the operations every game must support, taken from the RAD operation contracts: start(),submitAnswer(String), endGame() and getScore(). The abstract class *Game* implements it, and Jumble and WordLadder extend *Game.*
- **GameObserver**: one method, update(GameEvent event). Implemented by JumbleController and WordLadderController so the model can tell the screen to refresh without depending on JavaFX.
- **PuzzleSource**:nextPuzzle(GameType, Difficulty). Implemented by PuzzleBank, which holds the word lists for each difficulty. A different word source could replace it without changing either game.

Type Signatures and Visibility

- All fields are private, Getters are added only when another class needs the value. There are no public setter for state, so the state changes only through game operations such as submitAnswer().
- Helper methods such as scrambleWord() and checkAnswer() are private or protected. Members that Jumble and WordLadder share through Game (for example attemptsLeft and loadNextPuzzle() are protected.
- Fixed sets of values use enums instead of strings: *GameType* { JUMBLE, WORD_LADDER}, *Difficulty* {EASY, MEDIUM, HARD}, *GameStatus*{ ACTIVE, COMPLETED} and GameEvent {PUZZLE_READY, CORRECT, INCORRECT, OUT_OF_ATTEMPTS }. This refines the RAD class diagram, where gameStatus was a String, and stops invalid values like “Active at compile time.
- Collections are returned as unmodifiable copies (List.copyOf(games)) so callers cannot change model state directly.

Preconditions, Postconditions and Invariants

Every public method has Javadoc(*@param,@return, @throws*) with @pre and @post tags. The contracts below carry the RAD operation contracts down to the design-level methods. Breaking a precondition is a programming error and throws *IllegalAgrumentException* or *IllegalStateException*. A wrong answer from the player is input, not an error, so it is reported through a *GameEvent* instead.

Operation	Precondition	Postcondition
GameSystem.startGame(GameType type, Difficulty d)	No game is active; type and d are not null	A new Game exists with gameStatus == ACTIVE, the game type stored, a new Score of 0, and a random theme and music applied
Jumble.playJumble()	The game is ACTIVE	currentPuzzle is set; scrambledWord is a rearrangement of correctWord and not equal to it; attemptsLeft == MAX_ATTEMPTS
WordLadder.playWordLadder()	The game is ACTIVE	startWord and targetWord come from the puzzle; currentWord == startWord; attemptsLeft == MAX_ATTEMPTS
Game.submitAnswer(String answer)	The game is ACTIVE; a puzzle is displayed; answer is not blank	Correct: points increase and corrects goes up by 1. Incorrect: points unchanged, incorrects goes up by 1 and attemptsLeft drops by 1. At 0 attempts, the answer is shown and the next puzzle is loaded
GameSystem.viewScore()	A game session exists	Returns the current points, corrects and incorrects; game state is unchanged
Game.endGame()	The game is ACTIVE	gameStatus == COMPLETED; the final score is calculated and stored in Player.currentScore

Invariant(must be true before and after every public method call):

- *Game*: gameStatus is ACTIVE or COMPLETED, and 0 <= attemptsLeft <= MAX_ATTEMPTS.
- *Jumble*: scrambleWord always contains exactly the same letters as correctWord.
- *WordLadder*: startWord, currentWord and targetWord are the same length, and every accepted word differs from the previous one by exactly one letter.
- *Score*: points, correct and incorrect answers are never negative.
- *GameSystem*: only one instance exist(Singleton), and at most one game is ACTIVE at a time.

Inheriting Contracts

Subclasses must keep the contracts of the types they extend or implement(the Liskov Substitution Principle). *Jumble* and *WordLadder* may not add preconditions that *Game* does not have, may not weaken its postconditions, and must keep all of its invariants. For example, *WordLadder* checks an answer differently from *Jumble* (one letter change and a real word, versus an exact match). If the word is not valid, though, *WordLadder*.*submitAnswer*() must still report *INCORRECT* and use up an attempt, as *Game* promises. It must not throw an exception.This guarantees that a *GameController* written against *Game* works the same way with both games.

2 Design Specifications

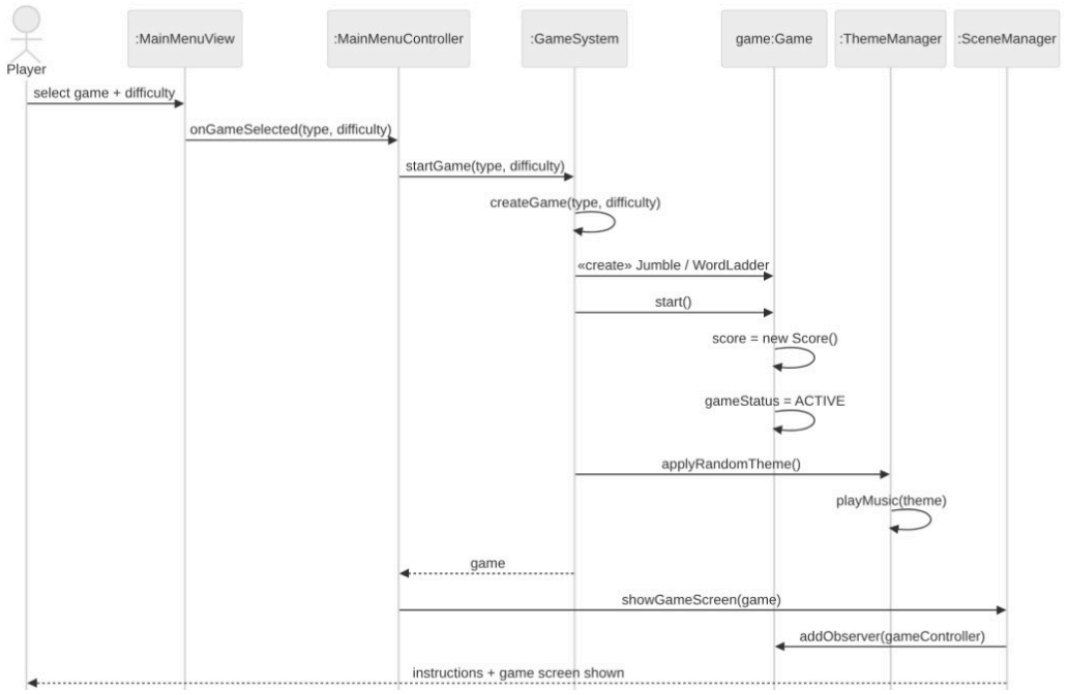
2.1 Use Case Realizations

Hand draw or tool-based UML sequence diagrams illustrating realizations of use case scenarios.

You need to perform use case realization on the **5 SSDs** that you were supposed to create in the Requirements and Analysis Document.

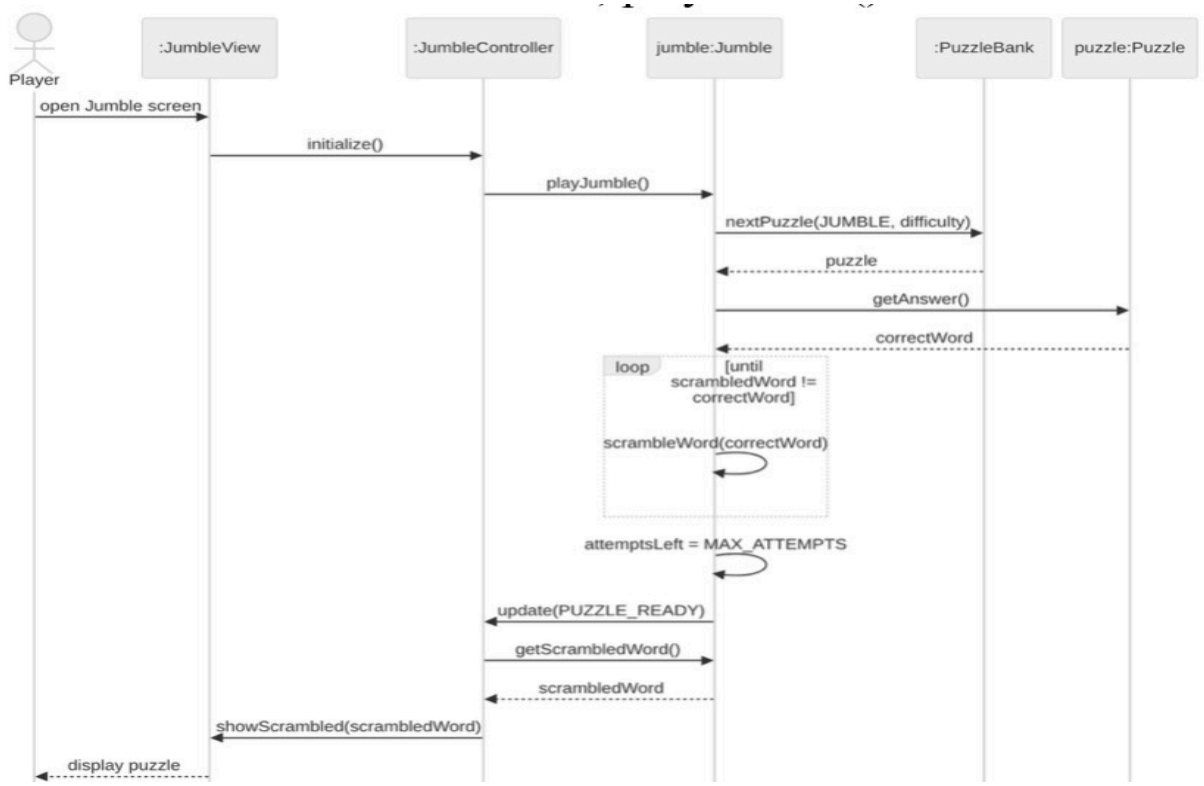
Remember, use case realization is the process of extending a **black-box SSD** to a **white-box SSD**. You need to think about all the objects that will play a role in a use case. Using that knowledge, you can create your “design-level” class diagram below.

Use case 1: Start a Game
The player chooses Jumble or word Ladder and a difficulty on the main menu. MainMenuController passes the choice to GameSystem(Singleton), which uses a factory method to create the right



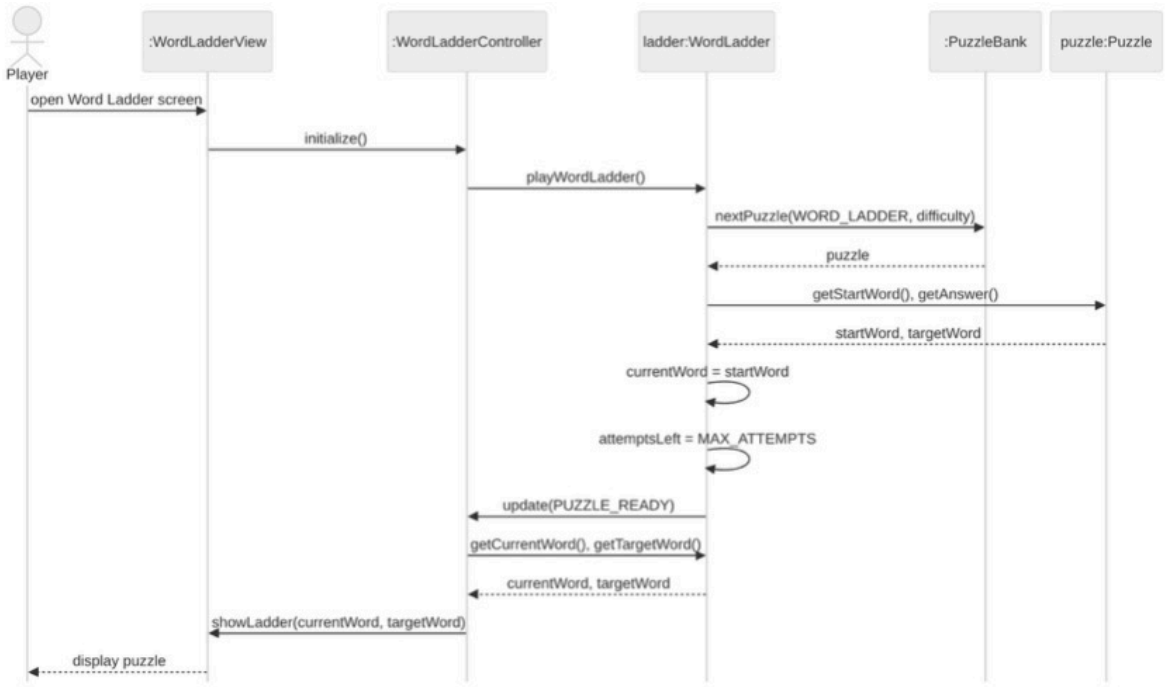
Use Case 2: Play Jumble

When the Jumble screen opens, JumbleController calls playJumble(). Jumble gets a puzzle for the chosen difficulty from Puzzlebank, reads the answer from the Puzzle and scrambles it until the scrambled word differs from the answer. It then resets the attempts and notifies the controller, which shows the scrambled word. Realizes SSD 2, playJumble().



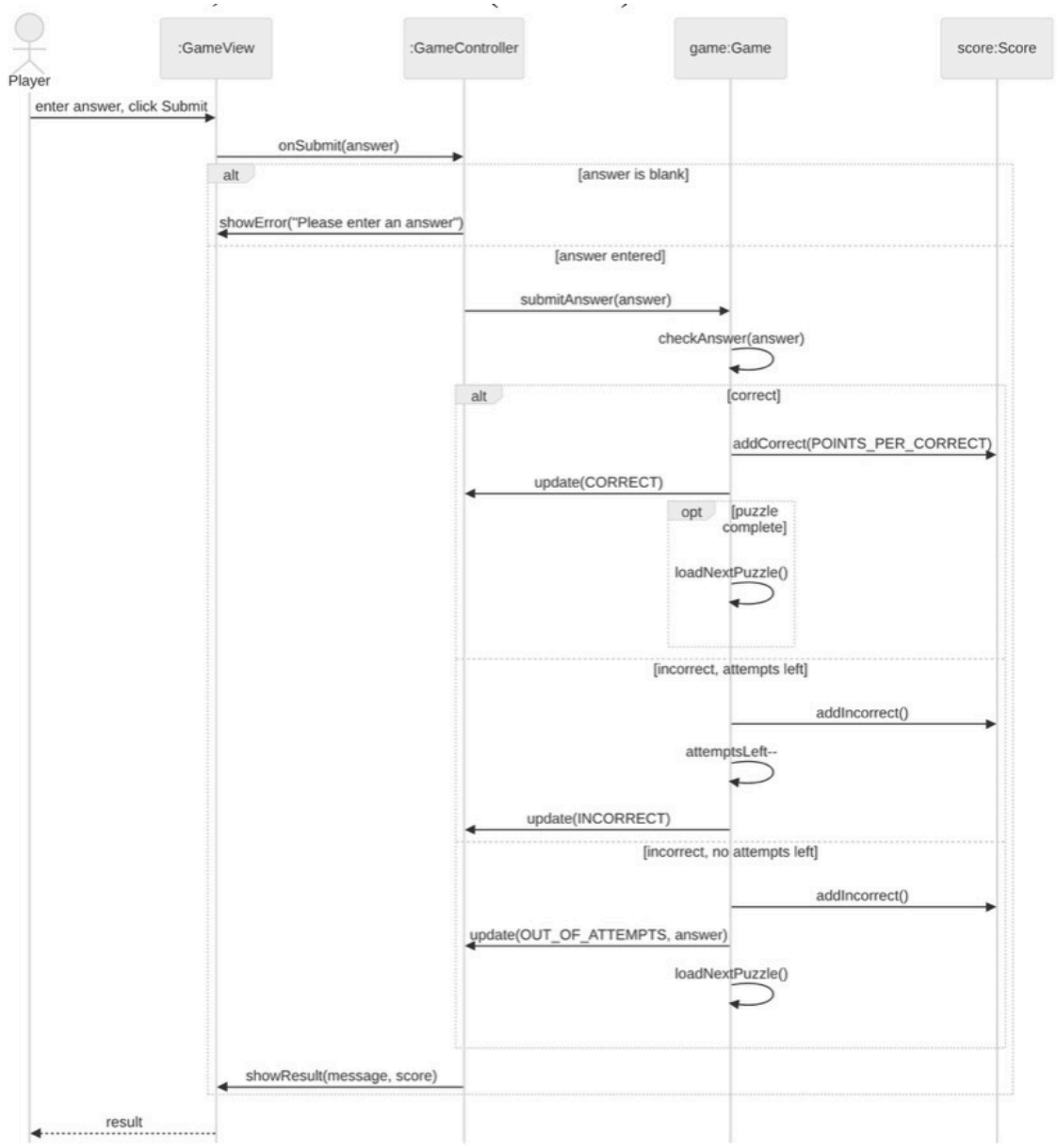
Use Case 3: Play Word Ladder

When the Word Ladder opens, *WordLadderController* calls *playWordLadder*().*WordLadder* gets a puzzle from *PuzzleBank* and reads it start and target words. It sets the current word to the start word, as Operation Contract requires. The controller is notified and displays both words. Realizes SSD 3, playWordLadder().



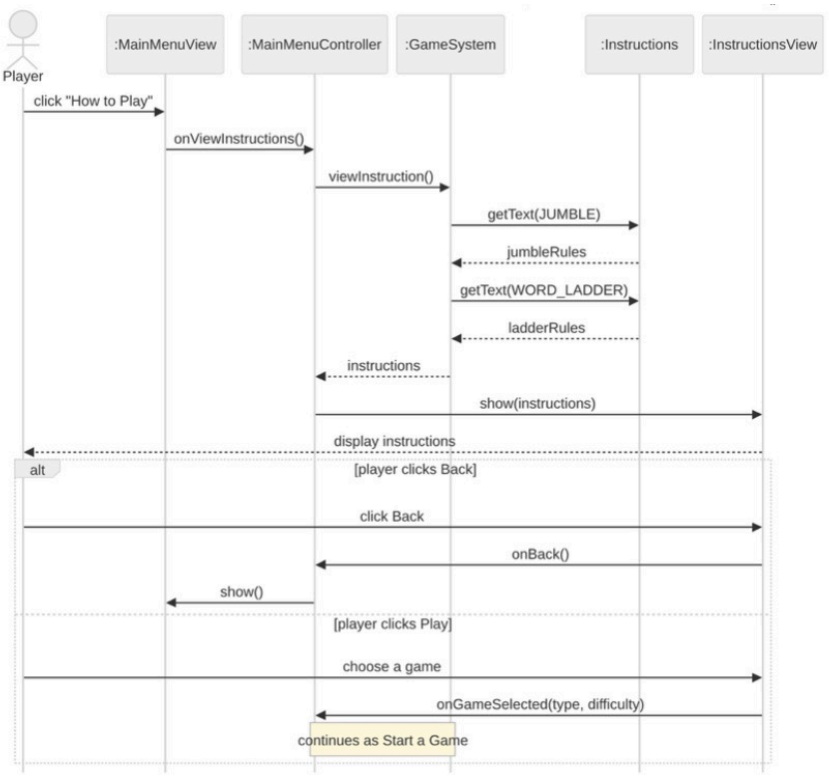
Use Case 4: Submit an Answer

The controller rejects a blank answer before it reaches the model(the invalid input requirement). Otherwise the game checks the answer: Jumble compares it with the correct word, and WordLadder checks that it is a real word one letter away from the current word. A correct answer adds points and may complete the puzzle, An incorrect answer is counted and uses an attempt. When no attempts are left, the correct answer is shown and the next puzzle loads, following the RAD's alternative flow. In every case, the controller shows the result and the updated score. Realizes SSD 4, *submitAnswer*(answer)



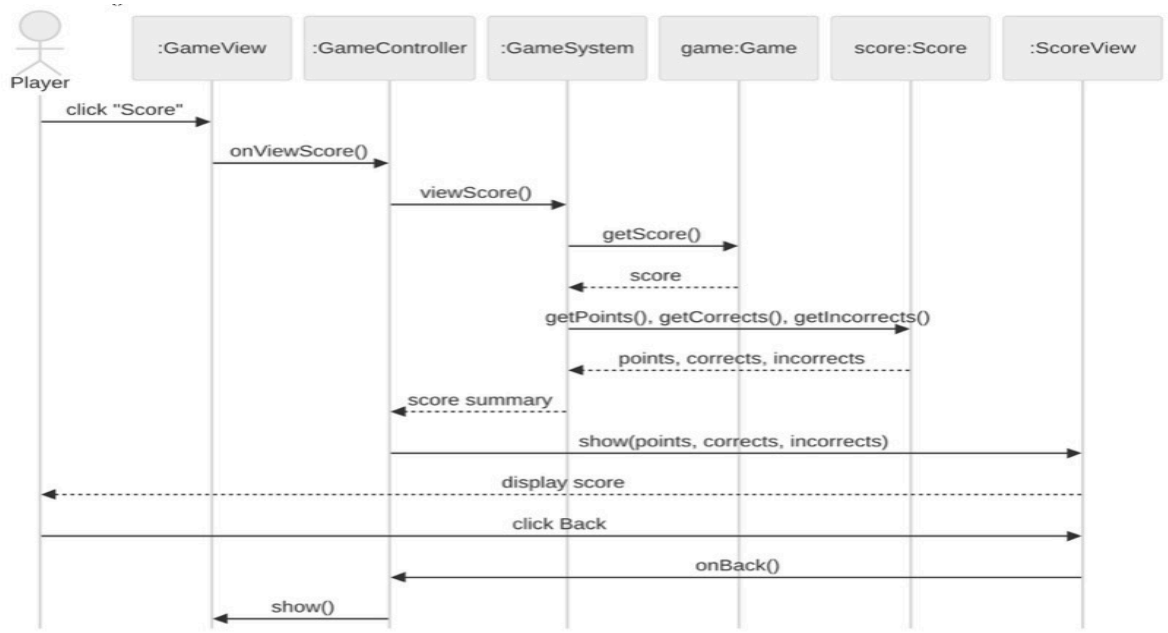
Use Case 5: View Instructions

The player clicks “How to Play” on the main menu. GameSystem collects the rules for both games from Instructions, and the controller displays them. Following Operation Contract 5, the player can then go back to the main menu or pick a game, which continues as Use Case 1. Realizes SSD 5, viewInstruction().



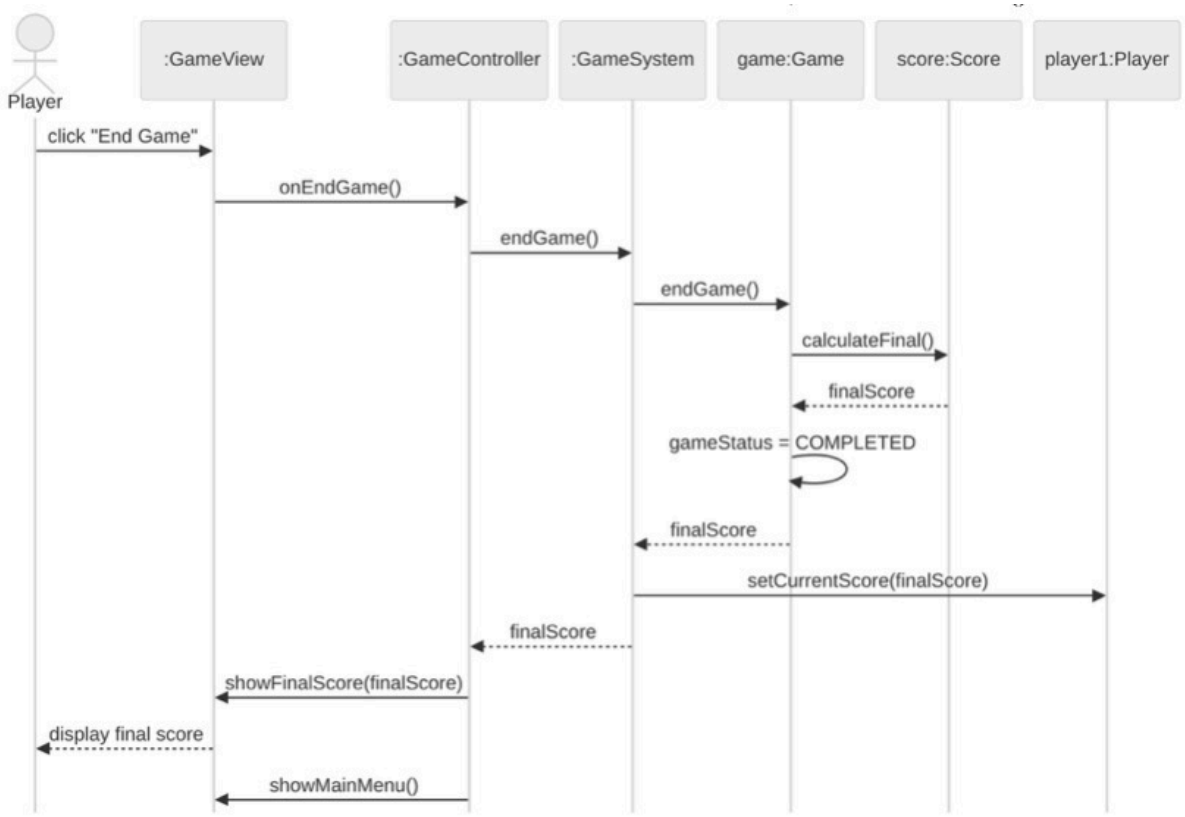
Use Case 6: View Score

GameSystem gets the active game's Score and reads its points, correct answers and incorrect answers. The controller shows them on the score screen. When the player clicks Back, they return to the game they were playing. Realizes SSD 6, viewScore().



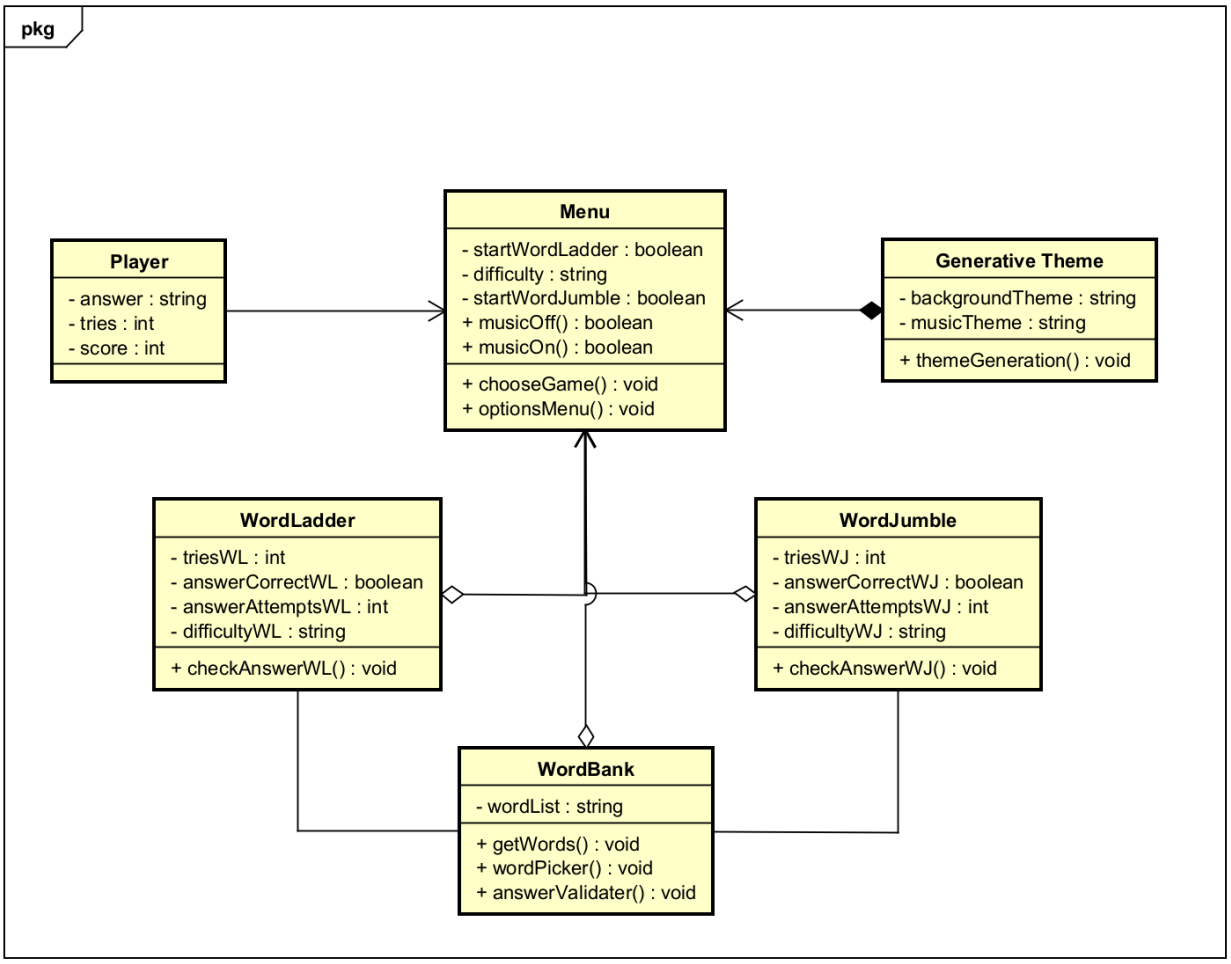
Use Case 7: End Game

The player clicks “End Game.” The game asks its Score for the final total and changes its status to COMPLETED. GameSystem stores the result in the Player object’s currentScore.The controller shows the final score and returns the player to the main menu, which meets the postconditions of operation Contract 7. Nothing is written to disk, because the RAD states that no data is saved between sessions. Realizes SSD 7, endGame().



2.2 Design Class Diagram

This is where you put the above mentioned “design-level” class diagram.



2.3 Major Patterns

The Word Games Application will be using the Model-View-Controller pattern to organize and separate the data we will be using such as the user interface and the logic within the program. The Model will manage both the data and actual functionality of Word Games and Jumble. The View is responsible for displaying the actual information to the player such as instructions, game content, and actual results of the game. The Controller will be the mediator between the two and is responsible for handling the inputs from the player and communicating to both the Model and View to process the actions occurring and updating the game to fit them.

The Observer design pattern is used to oversee and manage the communication between our different parts within the program. For example, if something were to change in the game such as the user inputting an answer in one of the games or even finishing a game, the appropriate

parts can be notified so that the information that needs to be shown to the player is correctly updated and shown to the player. This also will allow us to not rely so heavily on direct dependencies between certain parts of the program.

All in all, patterns such as these are implemented in order to improve the organization and maintainability of the program. This also allows for the program to be modified and expanded upon much easier as we do not need to go in and make a major change to the entire program every time
we plan to add or remove something in our program.